

Lecture 5

Object-Oriented Programming and Design

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Object-Oriented Programming

- **Object-oriented programming:** Programming paradigm in which programs are *networks of message-sending objects*
- There is **no** universally accepted definition
 - For context: [\[1\]](#) [\[2\]](#) [\[3\]](#)
- Each object addresses a *concern* and ideally has a *single responsibility/role*)
 - anti-pattern: [God Class](#)

Tell, Don't Ask

Imagine you're a customer at a restaurant...

```
class Customer:  
    # ...  
    def make_order(self, waiter: Waiter):  
        waiter.take_order(...)
```

Tell, Don't Ask

Imagine you're a customer at a restaurant...

```
class Customer:
    # ...
    def make_order(self, waiter: Waiter, chef: Chef):
        waiter.fix_table(...)
        waiter.serve_water(...)

        chef.heat_pan(...)
        chef.cook(...)
```

Which design makes more sense?

Tell, Don't Ask

- **Message:** request from sender to receiver to do something

```
class Customer:  
    # ...  
  
    def make_order(self, waiter: Waiter):  
        waiter.take_order(...)
```

Tell, Don't Ask

- *Sending a message* is technically the same as *calling a method*, but there are slightly different implications:
 - *call a method*: sender might know implementation details of receiver
 - *send a message*: there is a gap between sender's intention and receiver's implementation
 - implies *independence* between senders and receivers
- Implementation details of receiver should be *abstracted* from sender

Object Internals & Public Interfaces

- In Python, all attributes/methods that start with a single `_` are taken to be *private*
 - Do not access these from outside
 - Python has no *built-in* way to disallow accesses to fields of a class from outside the class
- **Public interface:** set of public attributes and methods of an object
 - Always interact with an object through its *public interface* (why?)

Object Internals & Public Interfaces

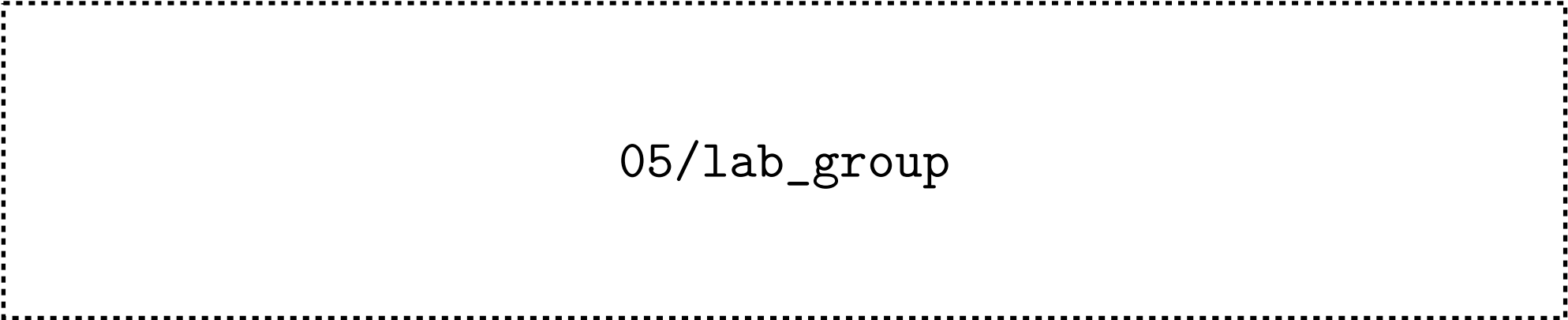
- `@property` makes a method that is accessed like an attribute:

05/property

- `@property` makes an *“immutable” attribute*

Maintaining Object Invariants

- **Invariant** (of an object): property that object promises to always hold
- Consider the following object that represents a single lab group:



05/lab_group

Maintaining Object Invariants

- Invariants: nonempty, at most two, no duplicates
- What are ways code outside LabGroup can use LabGroup's interface to have it violate its own invariants?

```
members = {'Yuuka', 'Noa'}
group = LabGroup(members)
print(group.members)          # {'Yuuka', 'Noa'}
members.add('Koyuki')
print(group.members)          # {'Yuuka', 'Noa', 'Koyuki'}
group.members.clear()
print(group.members)          # set() :(
```

OOP Buzzwords

- “Four pillars of OOP”
 - abstraction
 - encapsulation
 - polymorphism (subtype polymorphism)
 - inheritance

Only inheritance (future lec) is unique to OOP

Software Evolution

- The promise of OOP: code for which *change is easy to manage*
- Programs are created to fulfill client requirements
- *Requirements may change over time*
 - Source code may have to be changed as a result
- How do we make change easy to manage?
 - We can't predict the future!

Motivating Example

This is code for a card game:

```
05/card_game
```

What can you say about this implementation?

Composition

- **Composition:** objects containing *references* to other objects
 - Introduces a dependency between A and B (cannot make A without B)
 - A *has a* B

Composition

```
class B:
    ...

class A:
    def __init__(self, ...):
        self.b: B = ...

@dataclass
class A:
    b: B
```


Dependency Injection

- **Dependency Injection (DI):** parameterizing of dependencies in constructor
- Dependency that is “injected” (loosens coupling, more flexible):

```
class A:  
    def __init__(self, b: B):  
        self.b = b      # A can be made with any kind of B due to DI
```

Static Factory Method

- **Static method:** method that belongs to a class
- **Static factory method:** static method of a class that returns an instance/object of the class

Static Factory Method

- More convenient instantiation with default arguments (especially for DI classes):

05/static_factory

- Minor advantage over initializers: Can return **None** on failure due to invalid arguments

Forwarding

- **Forwarding:** Passing of message to another object
- Usually used with composition:

```
class A:
    def __init__(self, b: B):
        self.b = b

    def do_work(self, n: number): # A handles `do_work` "message" by
        return self.b.do_work(n) # forwarding same message to B and
                                # returning whatever it returns
```

- Allows code reuse with composition (later)

Demo

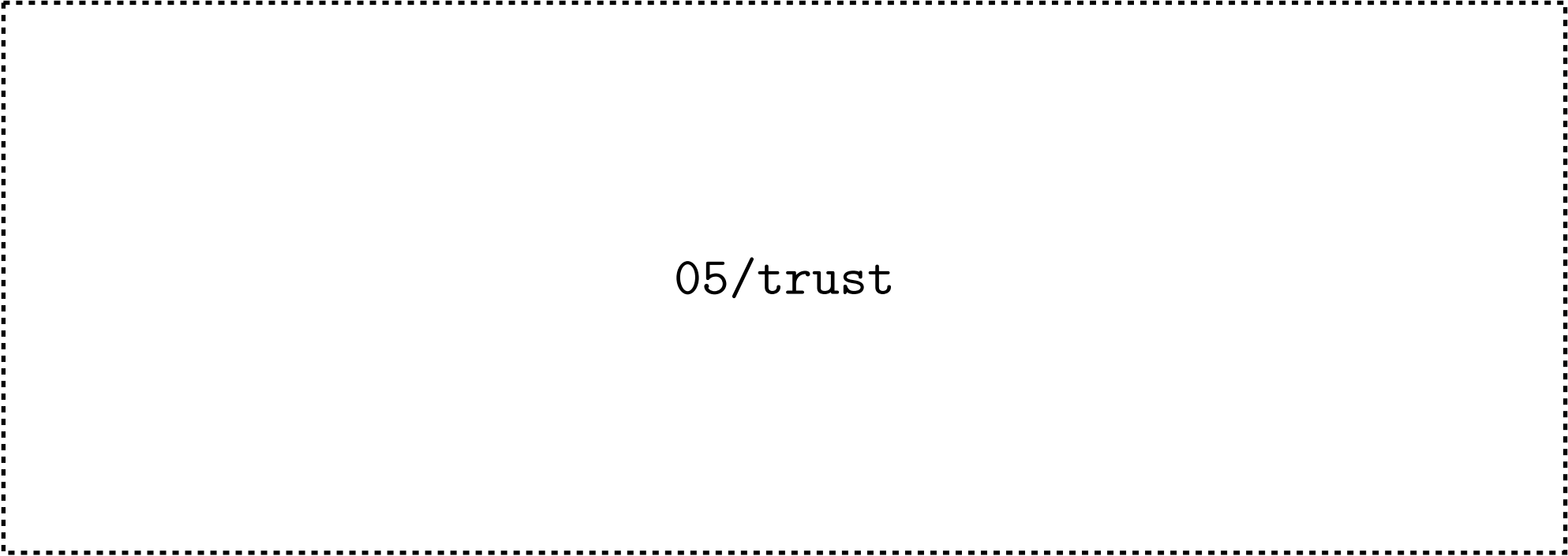
- **Task:** Simulate Part 2 (n rounds) of *Evolution of Trust* (<https://ncase.me/trust>) with unit tests and separation of concerns
- Copycat: Does last action of opponent (Cooperate initially)
- All Cooperate: Always cooperates
- All Cheat: Always cheats
- Grudger: Cooperate until cheated on, then Cheat
- Detective: Cooperate, Cheat, Cooperate 2x, then Copycat if never cheated on, else Cheat

Demo

- Prisoner's dilemma matrix:

(+A, +B)	B cooperates	B cheats
A cooperates	(+2, +2)	(-1, +3)
A cheats	(+3, -1)	(+0, +0)

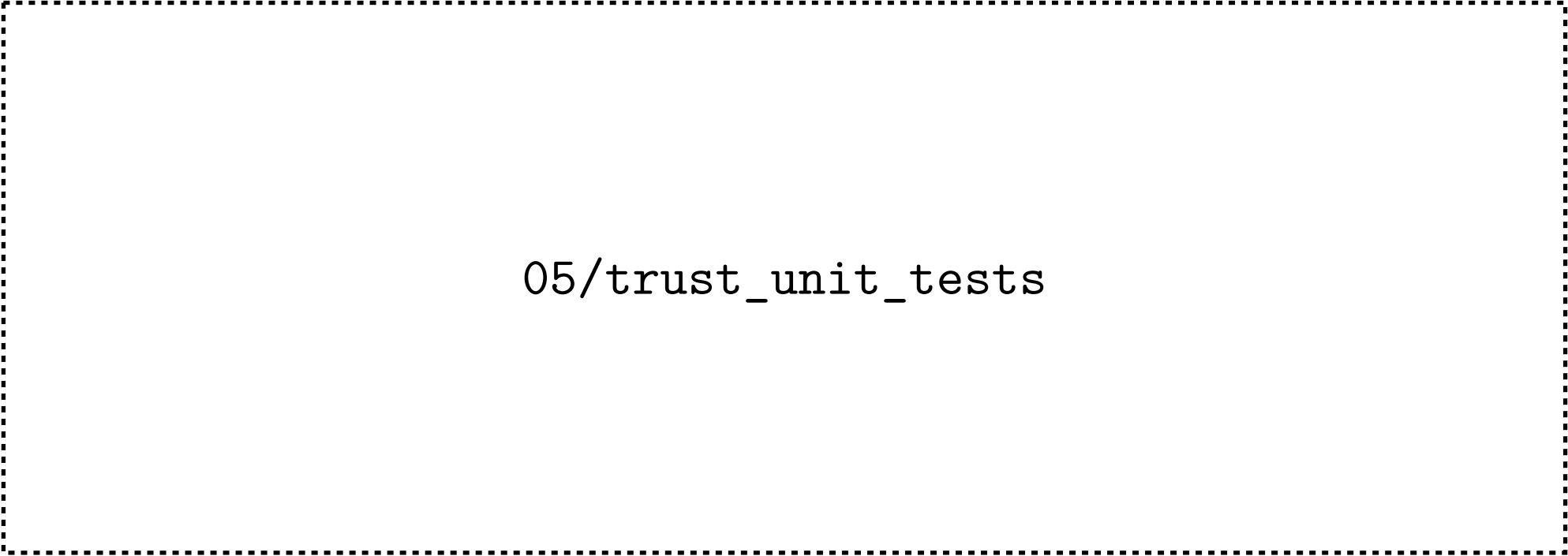
Possible Implementation



05/trust

Please study this carefully!

Demo: Unit Tests



05/trust_unit_tests

Ideally a lot more unit tests than just these

Design Trade-offs

Less coupled + more cohesive objects
⇒ Easier to manage change

- Fewer objects ⇒ Less coupled, less cohesive
- More objects ⇒ More cohesive, more complex, more overhead
 - Effect on coupling depends on design
- Aspects of code
 - Ease of coding, understandability, extensibility, performance

Guidelines

- Maximize *cohesion*, minimize *coupling*
 - Gather together things that change for the same reason
 - Separate those that don't
- Remove unnecessary dependencies
 - Objects should not know "too much" about others
- Keep classes and methods "small"